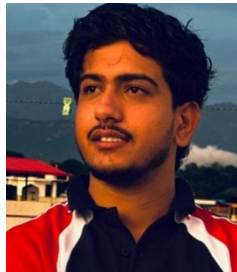


PROJECT AND TEAM INFORMATION

Project Title

Railway Ticketing Simulator

Student Information

<i>Team Name: Void Main</i> <i>Team # IDS-III-2026-T190</i>	
Team member 1 (Team Lead) <i>Aditya Singh: 2510320517</i> <i>iamadityasingh29@gmail.com</i>	
Team member 2 <i>Animesh Kumar Pandey: 2510320548</i> <i>Animeshhp19@gmail.com</i>	
Team member 3 <i>Arnav Kumar: 2510320475</i> <i>2510320475@geu.ac.in</i>	
Team member 4 <i>Purushottam Tawaniya: 2510320540</i> <i>2510320540@geu.ac.in</i>	

PROPOSAL DESCRIPTION

Motivation

Everyone who has booked a train ticket in India is aware of the anxiety involved in receiving a "Waitlist" ticket. Once you find yourself on the waiting list, the one question that comes to mind is straightforward: should they pack their bags or should they look for a bus?

At the moment, passengers have to guess. For example, you might see a number such as 'WL 8', but you won't know what it actually means in terms of your journey. Does that mean that eight people will cancel? Or is your seat likely to become available, or are you simply wasting your time by waiting?

The aim of this project is to make a passenger's life easier by instead of providing a general waiting number it instantly works out your real chance of securing a confirmed seat, expressed as a simple percentage. This enables ordinary travelers to make fast and realistic decisions regarding their journey with no confusion.

State of the Art / Current solution

Today's passenger is typically faced with two primary choices:

1. **Official Apps (IRCTC):** They tell you your current position (for example, WL 5) but do not tell you anything about your actual chances of confirmation.
2. **Third-Party Apps:** These try to guess what your chances are. However, they depend on complex background checks that are hard to understand and often inaccurate for quick analyzes.

Our project is direct and transparent, which solves this. It handles 10 confirmed seats, and as soon as the train fills up, it automatically puts additional passengers on a wait list, and tells them upfront "You are WL #2, and you have an 80% chance of getting confirmed".

Project Goals and Milestones (2 pts)

Overall Objective: I want to use a lot of fundamental C concepts that I have been practicing in our class- Structures, Arrays, Pointers, and Queues-to make a simple but robust train reservation program. This program would have 10 guaranteed seats, position it to take care of overflows using the waiting list, and instantly calculate and show wait-listed passengers how likely they are to get a seat when the program prints out their ticket.

Key Milestones:

- **Milestone 1:** (Booking & Confirmed seats): I will build the terminal menu and a static list of 10 guaranteed seats that utilize a Passenger structure. This list will be able to handle simple inputs like name and age, and the PNR for the main booking program.
- **Milestone 2:** (Waiting list setup): I will build out a first-in, first-out (FIFO) Waiting Queue of Passenger nodes to allow passengers to be placed in line when all 10 seats are filled. When the queue fills up, the next passenger will be put in the waiting list automatically.
- **Milestone 3:** (Confirmation chances): I will come up with a quick formula to produce each wait-listed passenger's precise percentage of confirmation before printing out their ticket.

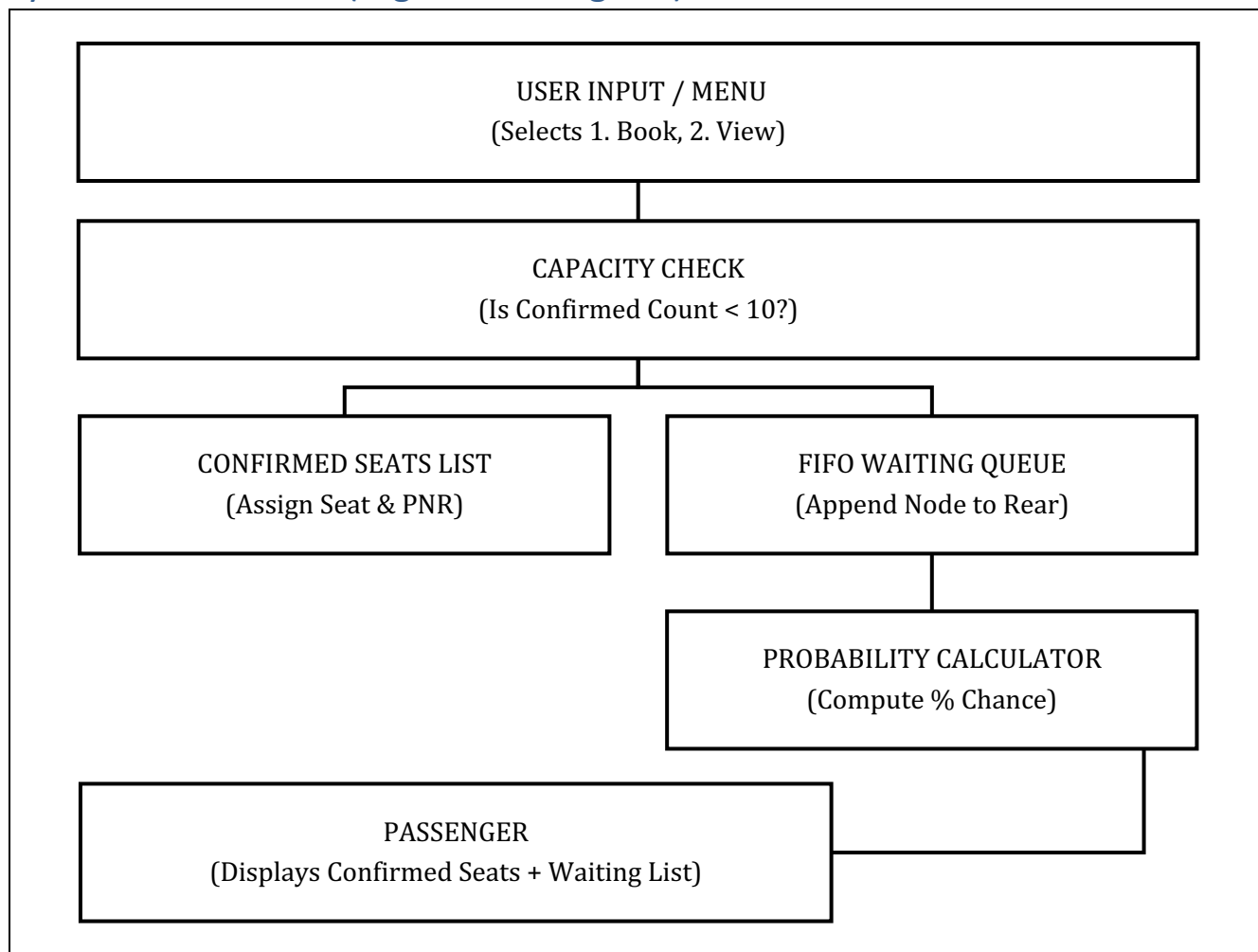
Project Approach (3 pts)

Our project will be done in C using VS Code, GCC compiler. The application will use a simple terminal window with a clear menu loop switch-case. To make the code more sleek and bug free, we are storing static things (the 10 fixed seats) in a simple array and using ready-made dynamic lists for the waiting queue.

- Passenger Information: We use a struct Passenger with PNR, Name, Age. Confirmed seats will go in an array of 10 seats.
- Waiting list logic: As soon as the list of 10 seats gets full, new additions go into Queue (front and rear pointers). First person to come on the waiting list gets the opportunity first when the seat is available.
- Confirmation Odds: In order for it to be presented to the user as something useful, we perform the confirmation calculation as a simple mathematical equation:

$$\text{Chance (\%)} = (1.0 - \text{Your Waiting Position} / \text{Total Seats}) \times 100$$
- Interface: a nice, tidy console window which separates 2 zones: the "Confirmed Passengers" zone on the top, and the "Waiting List with Confirmation Chances" zone below.

System Architecture (High Level Diagram)



Project Outcome / Deliverables

As this project is the creation of a fully operational C Console Application, the major goal and immediate goal of this project will be to build and test a functional, real-time Console-based ticket-manager and Queue-simulator, which meets the specifications.

The deliverables include:

- *Passenger Dashboard* - A real time console-based player performance dashboard, *Dual-View Rosters* - that will show both the 10 Confirmed Seats and the Watching List in real time, Clearly highlight booking statuses to keep passengers informed
- *Dynamic Watching Engine* - List logic that will kick in when the 10 Seat Capacity is reached, seamlessly transition bookings into a FIFO Queue, keep passenger details consistent and consistent without crashing.
- *Predictive Analytics* utilities like:
 - *Confirmed chance predictor* - Will accurately predict, in real time, the chance that the new customer (placed in the waiting line) will get seat-placed or not,
 - *Live queue position* - Will generate a waiting number (e.g., WL #1, WL #2) real-time for each customer
- *Interactive input loop* -A C control switch (switch-case) driven CLI (Command Line Interface) that lets the user continually book tickets, constantly view real-time status updates and test queue calculations in one sitting.

Assumptions

1. Valid Inputs: The user must enter a valid name and number when booking.
2. No more than 10: The train coach has only 10 confirmed seats for the first version of the app.
3. Confirmed seats: Tickets are sold on a first come, first served basis.
4. Local app: The terminal runs the app locally for testing and assessment purposes.

References

- *GeeksforGeeks. "Queue Data Structure in C."*
 - Available at: [<https://www.geeksforgeeks.org/queue-data-structure/>]
- *Programiz. "C Structures (struct)."*
 - Available at: [<https://www.programiz.com/c-programming/c-structures>]
- *W3Schools. "C Pointers and Dynamic Memory."*
 - Available at: [https://www.w3schools.com/c/c_pointers.php]